

CARLOS EXPEDITO RODRIGUES ARAGÃO

POSTECH

IA PARA DEVS

PROCESSAMENTO DE LINGUAGEM NATURAL

AULA 05

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	10
O QUE VOCÊ VIU NESTA AULA?	16
REFERÊNCIAS.....	17

EMENDAS

O QUE VEM POR AÍ?

Bem-vindos e bem-vindas! Nesta aula, você vai criar um projeto novo de classificação de categorias de títulos de notícias em um dataset e vai aprender os conceitos poderosos de Word Embedding com a arquitetura Word2Vec.



HANDS ON

Implementaremos o Word2Vec em um novo projeto de classificação de categoriais de títulos de notícias. Os arquivos csv estarão disponíveis na plataforma.

Primeiro, vamos descompactar os arquivos zip que baixamos no site do NILC.

```
!unzip "/content/drive/MyDrive/Colab Notebooks/Word2Vec/cbow_s300.zip"
```

```
!unzip "/content/drive/MyDrive/Colab Notebooks/Word2Vec/skip_s300.zip"
```

Importar os datasets pelo drive, não se esqueça de montar a conexão com o drive no seu colab.

```
import pandas as pd

artigo_treino = pd.read_csv("/content/drive/MyDrive/Colab
Notebooks/Word2Vec/treino.csv")

artigo_teste = pd.read_csv("/content/drive/MyDrive/Colab
Notebooks/Word2Vec/teste.csv")
```

Fizemos um exemplo de vetorização com one hot encoding.

```
from sklearn.feature_extraction.text import CountVectorizer

texto = [
    "Este produto é muito bom",
    "Este produto é muito ruim"
]

vetorizador = CountVectorizer()
vetorizador.fit(texto)
```

```
print(vetorizador.vocabulary_)
```

```
{'este': 1, 'produto': 3, 'muito': 2, 'bom': 0, 'ruim': 4}
```

```
vetor_bom = vetorizador.transform(["bom"])  
print(vetor_bom.toarray())
```

```
[[1 0 0 0 0]]
```

Abrir o arquivo chave vetor com o gensim.

```
from gensim.models import KeyedVectors  
modelo = KeyedVectors.load_word2vec_format("cbow_s300.txt")
```

Criando método tokenizador:

```
import nltk  
import string  
nltk.download('punkt')  
  
def tokenizador(texto):  
    texto = texto.lower()  
    lista_alfanumerico = []  
  
    for token_valido in nltk.word_tokenize(texto):  
        if token_valido in string.punctuation: continue  
        lista_alfanumerico.append(token_valido)  
  
    return lista_alfanumerico
```

Criando método combinacao_de_vetores_por_soma.

```
import numpy as np

def combinacao_de_vetores_por_soma(palavras_numeros):
    vetor_resultante = np.zeros(300)
    for pn in palavras_numeros:
        try:
            vetor_resultante += modelo.get_vector(pn)
        except KeyError:
            if pn.isnumeric():
                pn = "0"*len(pn)
                vetor_resultante += modelo.get_vector(pn)
            else:
                vetor_resultante += modelo.get_vector('unknown')
    return vetor_resultante
```

Criando método matriz_vetores.

```
def matriz_vetores(textos):
    x = len(textos)
    y = 300
    matriz = np.zeros((x,y))

    for i in range(x):
        palavras_numeros = tokenizador(textos.iloc[i])
        matriz[i] = combinacao_de_vetores_por_soma(palavras_numeros)
```

```
return matriz
```

Criando matrizes das nossas bases.

```
matriz_vetores_treino = matriz_vetores(artigo_treino.title)
matriz_vetores_teste = matriz_vetores(artigo_teste.title)
```

Criando modelo e printando o classification report.

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report

LR = LogisticRegression(max_iter = 200)
LR.fit(matriz_vetores_treino, artigo_treino.category)
label_prevista = LR.predict(matriz_vetores_teste)
CR = classification_report(artigo_teste.category, label_prevista)
print(CR)
```

Comparando com o DummyClassifier.

```
from sklearn.dummy import DummyClassifier

DC = DummyClassifier()
DC.fit(matriz_vetores_treino, artigo_treino.category)
label_previsao_dc = DC.predict(matriz_vetores_teste)
CR_dummy = classification_report(artigo_teste.category, label_previsao_dc)
print(CR_dummy)
```

Abrindo arquivo chave vetor de skip gram.

```
modelo_skipgram = KeyedVectors.load_word2vec_format("skip_s300.txt")
```

Criando método `combinacao_de_vetores_por_soma_skipgram`.

```
def combinacao_de_vetores_por_soma_skipgram(palavras_numeros):  
    vetor_resultante = np.zeros(300)  
    for pn in palavras_numeros:  
        try:  
            vetor_resultante += modelo_skipgram.get_vector(pn)  
        except KeyError:  
            if pn.isnumeric():  
                pn = "0"*len(pn)  
                vetor_resultante += modelo_skipgram.get_vector(pn)  
            else:  
                vetor_resultante += modelo_skipgram.get_vector('unknown')  
    return vetor_resultante
```

Criando método `matriz_vetores_skipgram`.

```
def matriz_vetores_skipgram(textos):  
    x = len(textos)  
    y = 300  
    matriz = np.zeros((x,y))  
  
    for i in range(x):  
        palavras_numeros = tokenizador(textos.iloc[i])  
        matriz[i] = combinacao_de_vetores_por_soma_skipgram(palavras_numeros)  
  
    return matriz
```

Criando matrizes skip gram.

```
matriz_vetores_treino_skipgram = matriz_vetores_skipgram(artigo_treino.title)
matriz_vetores_teste_skipgram = matriz_vetores_skipgram(artigo_teste.title)
```

Criando modelo e printando o classification report do skip gram.

```
LR_skipgram = LogisticRegression(max_iter = 200)
LR_skipgram.fit(matriz_vetores_treino_skipgram, artigo_treino.category)
label_previsao_skipgram = LR_skipgram.predict(matriz_vetores_teste_skipgram)
CR_skipgram = classification_report(artigo_teste.category,
label_previsao_skipgram)
print(CR_skipgram)
```

SAIBA MAIS

O Word Embedding, uma técnica essencial no domínio do Processamento de Linguagem Natural (PLN), revoluciona a interpretação de palavras por máquinas. Vai além de simplesmente associar palavras a valores numéricos, buscando encapsular o significado semântico em um espaço vetorial contínuo.

O Word2Vec, desenvolvido pelo Google, é uma pedra angular nesse contexto. Sua arquitetura, com as variantes Skip-Gram e Continuous Bag of Words (CBOW), permite uma modelagem robusta das relações semânticas. Já o GloVe adota uma abordagem única, utilizando estatísticas globais de co-ocorrência para criar representações vetoriais que capturam nuances semânticas.

O FastText, criado pelo Facebook, vai além ao incorporar representações subpalavra. Decompondo palavras em n-gramas, captura a estrutura morfológica e semântica de maneira granular, útil especialmente para lidar com palavras menos frequentes ou idiomas complexos.

As aplicações são vastas. No PLN, o Word Embedding melhora tarefas como classificação de texto e análise de sentimento. Em sistemas de recuperação de informações, eleva a relevância dos resultados, compreendendo o contexto de busca. Em recomendação de conteúdo, proporciona sugestões mais precisas, enquanto em chatbots e assistência virtual, possibilita respostas mais naturais ao entender o contexto da conversa.

Mais do que uma técnica, o Word Embedding é uma ponte entre linguagem natural e aprendizado de máquina. Ao representar palavras em espaços contínuos, não apenas captura a riqueza semântica, mas também impulsiona avanços práticos, moldando o futuro do PLN e suas aplicações. Essa abordagem não só revoluciona como as máquinas entendem o significado das palavras, mas também redefine o panorama da inteligência artificial aplicada à linguagem.

O Word2Vec, uma das mais influentes inovações no campo do Processamento de Linguagem Natural (PLN), é uma técnica que redefine a maneira como as palavras são representadas e interpretadas por algoritmos de aprendizado de máquina. Desenvolvido pelo Google, o Word2Vec é notável por sua capacidade de capturar relações semânticas entre palavras em um espaço vetorial.

A arquitetura do Word2Vec é caracterizada por duas variantes principais: Skip-Gram e Continuous Bag of Words (CBOW). No modelo Skip-Gram, a ideia é prever as palavras do contexto com base em uma palavra de entrada. Por outro lado, o CBOW faz o oposto, utilizando o contexto para prever a palavra-alvo. Essa dualidade confere flexibilidade ao Word2Vec, permitindo que ele se adapte a diferentes contextos linguísticos.

O treinamento do Word2Vec envolve a criação de embeddings, ou seja, vetores numéricos para cada palavra do vocabulário. Esses vetores são ajustados de forma iterativa durante o processo de treinamento para que as palavras semanticamente similares estejam próximas no espaço vetorial. A similaridade entre vetores reflete a relação semântica entre as palavras, resultando em representações densas e ricas em contexto.

Uma das características notáveis do Word2Vec é sua capacidade de preservar analogias semânticas. Por exemplo, se considerarmos a relação "rei é para rainha, assim como homem é para mulher," o Word2Vec é capaz de inferir essa analogia a partir dos vetores associados a essas palavras.

Além disso, o Word2Vec demonstra eficiência mesmo em grandes conjuntos de dados, permitindo a criação de modelos de linguagem poderosos. Esses modelos treinados com Word2Vec têm implicações profundas em uma variedade de aplicações, desde a análise de sentimentos e classificação de texto até sistemas de recomendação e tradução automática.

O Word2Vec, ao proporcionar uma representação semântica significativa das palavras, não apenas impulsionou o avanço em tarefas específicas de PLN, mas também contribuiu para o desenvolvimento de modelos linguísticos mais sofisticados. Sua capacidade de extrair significado contextualizado de palavras estabeleceu um novo padrão na representação semântica, abrindo portas para uma compreensão mais profunda da linguagem natural por parte das máquinas. Assim, o Word2Vec continua a ser uma peça fundamental no quebra-cabeça do PLN, moldando e inspirando avanços contínuos na interseção entre linguagem e máquinas inteligentes.

O Global Vectors for Word Representation (GloVe) é uma técnica avançada de incorporação de palavras que se destaca no cenário do Processamento de Linguagem Natural (PLN) por sua abordagem inovadora, baseada em estatísticas globais de co-

ocorrência de palavras. Desenvolvido para capturar nuances semânticas complexas, o GloVe visa criar representações vetoriais que destacam as relações intrínsecas entre as palavras em grandes conjuntos de dados textuais.

A arquitetura do GloVe difere das abordagens mais tradicionais, como o Word2Vec. Em vez de depender exclusivamente de tarefas de predição direta, o GloVe incorpora informações estatísticas globais de co-ocorrência. O cerne do processo reside na construção de uma matriz de co-ocorrência, onde cada elemento indica a frequência com que duas palavras ocorrem juntas em um contexto específico.

Durante o treinamento, o GloVe ajusta os vetores de palavras iterativamente para minimizar a discrepância entre as representações vetoriais das palavras e as informações de co-ocorrência global. Esse método resulta em embeddings que preservam não apenas relações diretas, mas também hierarquias semânticas mais abstratas presentes nos dados.

A vantagem distintiva do GloVe é sua habilidade de lidar efetivamente com a escassez de dados e palavras menos frequentes, uma vez que se baseia em estatísticas globais. Essa característica o torna particularmente útil em cenários nos quais a coleta de dados suficientes para cada palavra é desafiadora.

Ao proporcionar representações vetoriais que refletem com precisão as relações semânticas entre palavras, o GloVe demonstrou eficácia em uma variedade de tarefas em PLN. Desde a análise de sentimentos até tradução automática e recuperação de informações, a capacidade do GloVe de capturar significados contextualizados tem impactado positivamente várias aplicações práticas.

A abordagem inovadora do GloVe, centrada na co-ocorrência global, não apenas expande o leque de técnicas de incorporação de palavras, mas também ressalta a importância de considerar informações estatísticas amplas para obter representações semânticas robustas. Assim, o GloVe permanece como uma ferramenta crucial na caixa de ferramentas do PLN, alimentando avanços contínuos na compreensão e interpretação da linguagem natural por parte das máquinas.

O FastText, uma inovação desenvolvida pelo Facebook, surge como uma abordagem poderosa na tarefa de incorporação de palavras (Word Embedding), indo além da representação convencional de palavras inteiras. Ao introduzir a análise de subpalavras, o FastText oferece uma perspectiva mais rica e detalhada da estrutura

morfológica das palavras, tornando-se uma ferramenta crucial no campo do Processamento de Linguagem Natural (PLN).

A arquitetura do FastText difere das técnicas tradicionais, como Word2Vec e GloVe, ao considerar subpalavras, ou n-gramas, no processo de treinamento. Cada palavra é desmembrada em partes menores, permitindo que o modelo capture informações não apenas sobre palavras completas, mas também sobre fragmentos morfológicos. Essa abordagem é especialmente valiosa para lidar com palavras menos frequentes, idiomas com morfologia rica e contextos nos quais a compreensão das partes constituintes é fundamental.

O treinamento do FastText envolve a criação de embeddings para subpalavras e palavras inteiras. Durante esse processo, o modelo ajusta os vetores iterativamente para representar de maneira eficaz a estrutura morfológica e semântica das palavras. Assim, o FastText produz representações vetoriais que incorporam não apenas o significado global da palavra, mas também suas partes constituintes.

Uma característica distintiva do FastText é sua capacidade de lidar com palavras fora do vocabulário, já que as subpalavras são compartilhadas entre diferentes termos. Isso torna o modelo robusto em situações em que o vocabulário é dinâmico ou a coleta de dados é desafiadora.

As aplicações do FastText são diversas e impactam positivamente uma ampla gama de tarefas em PLN. Em análise de sentimentos, classificação de texto, e até mesmo em tarefas mais complexas como tradução automática, o FastText se destaca ao compreender não apenas o significado geral das palavras, mas também suas nuances morfológicas.

Ao trazer a análise de subpalavras para o cenário de incorporação de palavras, o FastText representa uma evolução significativa na compreensão da linguagem natural por parte das máquinas. Sua abordagem inovadora destaca a importância de considerar a estrutura morfológica para obter representações semânticas mais refinadas. Assim, o FastText permanece como uma ferramenta essencial, expandindo os horizontes do PLN e contribuindo para avanços contínuos na interpretação da riqueza linguística que permeia nossa comunicação cotidiana.

O BERT (Bidirectional Encoder Representations from Transformers) emerge como um marco revolucionário no campo do Processamento de Linguagem Natural

(PLN), trazendo consigo uma abordagem inovadora que captura o contexto bidirecional das palavras. Desenvolvido pelo Google, o BERT tem redefinido os limites da compreensão de linguagem natural por modelos de aprendizado de máquina.

A arquitetura do BERT é baseada em transformers, uma classe de modelos que se destacam em tarefas de sequência, como tradução automática e resumo de texto. O BERT, no entanto, introduz um elemento chave: a atenção bidirecional. Ao contrário de modelos anteriores que processavam texto de maneira unidirecional, o BERT analisa o contexto em ambas as direções, levando a uma compreensão mais profunda e contextualizada.

Durante o treinamento, o BERT utiliza uma estratégia conhecida como masked language model (MLM). Parte das palavras em uma sentença é ocultada, e o modelo é treinado para prever essas palavras ocultas com base no contexto fornecido pelas palavras vizinhas. Esse método de treinamento aprimora a capacidade do modelo de entender o significado de cada palavra em relação ao contexto geral da sentença.

A capacidade única do BERT de entender o contexto bidirecional permite que ele lide com nuances semânticas e relações complexas entre palavras em uma sentença. Isso se traduz em um desempenho excepcional em uma variedade de tarefas de PLN, incluindo análise de sentimentos, classificação de texto, perguntas e respostas, e até mesmo em tradução automática.

Uma das contribuições mais notáveis do BERT é sua habilidade de lidar com ambiguidades e contextos polissêmicos. Ao considerar o contexto global de uma sentença, o modelo é capaz de discernir diferentes significados de uma palavra com base na forma como ela é utilizada na frase.

O BERT se tornou uma pedra angular em diversas aplicações práticas, sendo amplamente adotado em sistemas de busca, assistentes virtuais e chatbots. Sua arquitetura robusta e capacidade de compreensão contextual o tornam uma ferramenta essencial para as demandas crescentes de processamento de linguagem natural em aplicações do mundo real.

Em resumo, o BERT representa uma mudança de paradigma no campo do PLN, proporcionando uma compreensão mais profunda e contextualizada da linguagem natural. Sua arquitetura inovadora e abordagem bidirecional estabeleceram um novo

padrão para modelos de linguagem, destacando a importância do contexto global na interpretação precisa da linguagem.

EMAP

O QUE VOCÊ VIU NESTA AULA?

Você aprendeu os conceitos de Word2Vec e entendeu como funciona essa arquitetura de maneira mais profunda. Vimos como que palavras de contexto semelhante calculados da forma correta podem ajudar a fazer um classificador funcional.

EMEND

REFERÊNCIAS

GENSIM. **Documentação Gensim.** [s.d]. Disponível em: <<https://radimrehurek.com/gensim/models/keyedvectors.html>>. Acesso em: 17 jan. 2024.

MIKOLOV, T. CHEN, K. CORRADO, G. DEAN, J. **Efficient Estimation of Word Representations in Vector Space.** 2013. Disponível em: <<https://arxiv.org/pdf/1301.3781.pdf>>. Acesso em: 17 jan. 2024.

NILC. **Repositório de Word Embeddings do NILC.** [s.d]. Disponível em: <<http://nilc.icmc.usp.br/nilc/index.php/repositorio-de-word-embeddings-do-nilc>> Acesso em: 17 jan. 2024.

PALAVRAS-CHAVE

Word2Vec. Word Embedding. CBOW. SkipGram.

EMAP



POSTECH